Calc TDD 作業報告：人工 TDD 與 AI Agent TDD

項目	內容
學生	許家寧（學號 315552051）
語言／工具	Java 17 target · JUnit 4.13.2 · Maven · JaCoCo 0.8.12 · SpotBugs 4.8.6 · GitHub Actions
AI Agent（Q6、Q7）	Claude（Cowork），設定模型 ID `claude-opus-5-5`
期間	2026-10-22 ～ 2026-10-25（Asia/Taipei）
Repo	https://github.com/ning415415/calc-tdd

Question 1：以 TDD 加入 subtract、multiply、divide（不使用 AI coding agent）

1.1 git 歷史中的 TDD

#	Commit	類型	內容	紅燈型態	證據
0	`ae313b4`	chore	起始版（只有 `add`）＋ Maven、CI 設定	—	
1	`a5c68be`	test	`subtract(5,3) == 2`	編譯錯誤：`cannot find symbol: method subtract(int,int)`	§1.3 截圖

#	Commit	類型	內容	紅燈型態	證據
1	dd5927b	feat	加入 subtract(a - b)	綠：2 tests	§1.3 截圖
2	f60184a	test	subtract(2,3) == -1（負數結果）	無紅燈：一寫就通過	
2	4b6acb3	refactor	把負數斷言拆成獨立測試 testSubtractNegative	綠：3 tests	
3	c72087c	test	multiply(3,5) == 15	編譯錯誤	
3	75093c5	feat	加入 multiply	綠：4 tests	
4	1b8e430	test	int result = divide(7,2); result == 3	編譯錯誤	
4	1b761b5	feat	加入 divide（整數除法）	綠	
5	846a8fa	test	divide(7,0) 拋 ArithmeticException	無紅燈：Java 的 int 除以零本來就會拋	
6	287265e	test	divide(7,-2) == -3（往 0 截斷）	無紅燈	
—	047a85a	chore	拿掉 JaCoCo 行覆蓋率規則	—	

#	Commit	類型	內容	紅燈型態	證據
			（隱含建構子從未被呼叫）		

1.2 寫程式前用測試定下的設計決定

決定	選擇	由哪個測試定義
`divide` 回傳型別	`int`	`testDivide`：`7/2` 在 int 和 double 下結果不同；`int result =` 也在編譯期鎖住型別
除以零	拋 `ArithmeticException`（沿用 Java 行為）	`testDivideByZero`
負數除法	往 0 截斷：`7 / -2 == -3`	`testDivideNegtive`
減法結果為負	正常回傳負數	`testSubtractNegative`

1.3 每一輪的敘述

第 1 輪 subtract：

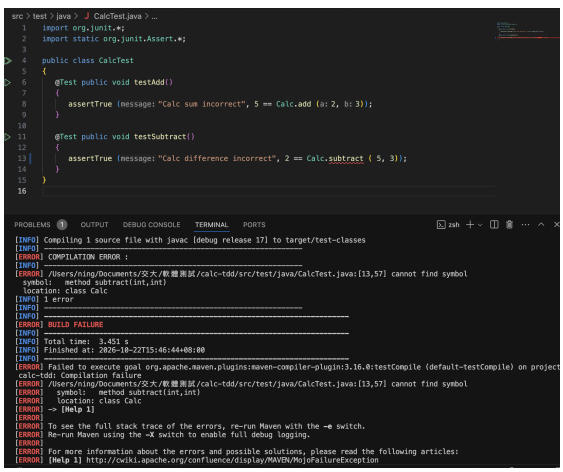
測試是否找得到subtract(int,int)

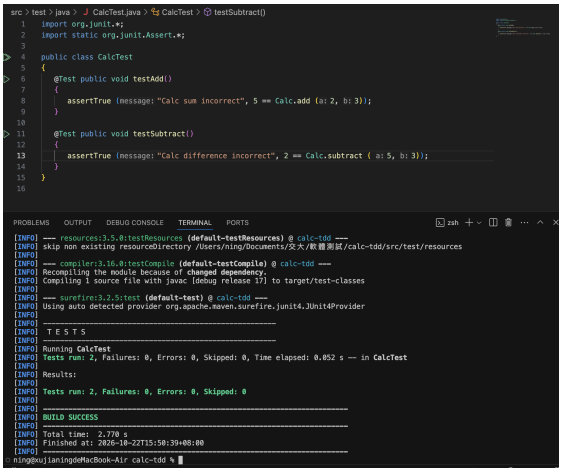
紅燈訊息：cannot find symbol: method subtract(int,int)

修改：到 Calc.java 加入 subtract。一開始把預期結果寫成5(5 == Calc.subtract(5, 3))，結果輸出必失敗，改成(2 == Calc.subtract(5, 3))

commit ：test: subtract returns difference (5,3)

commit ：feat: add subtract





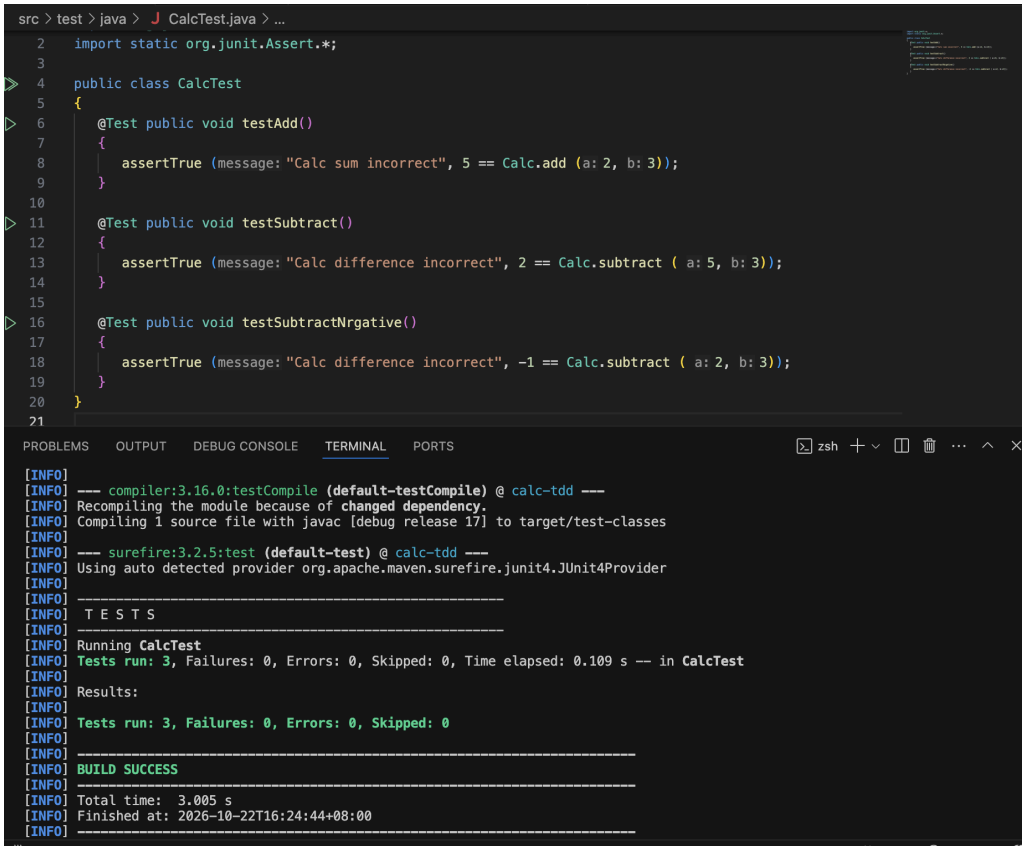
第 **2** 輪 **subtract** 負數結果＋重構：

我新增 testSubtractNegativeResult，確認小減大會得到負數。這個測試一加入就通過，因為上一輪的實作已經是 a - b，不需要修改程式碼。我保留它作為負數結果的規格。（截圖處名稱有錯字，之後有改為testSubtractNegative()）

Calc.subtract(5, 3))

commit test: subtract handles negative result (passes immediately)

重構：本來將負數測試寫在testSubtract下，但可能因為第一個測試未通過而尚未測試到第二個結果為負數的測試



第 **3** 輪 **multiply**:

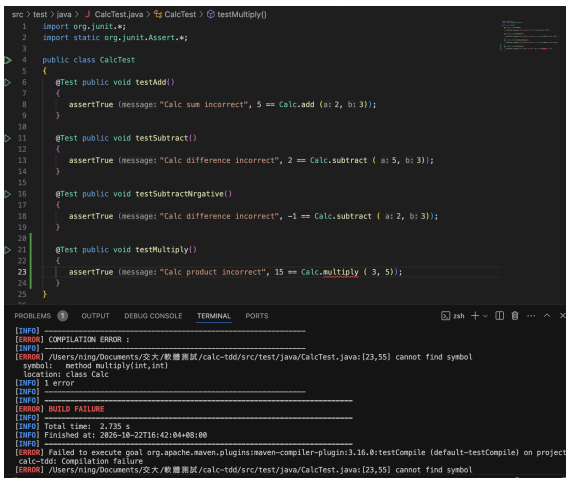
測試是否找得到multiply(int,int)

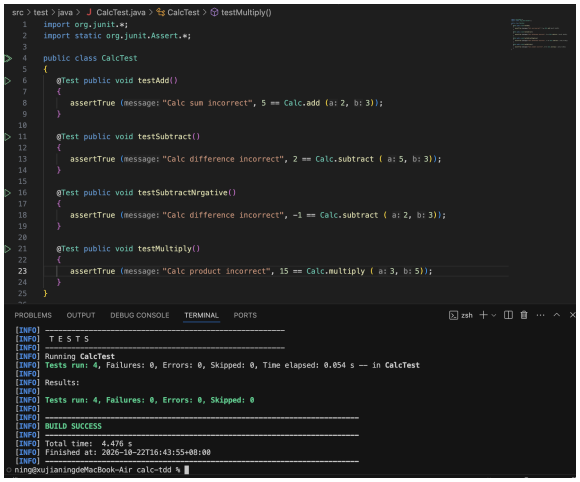
紅燈訊息:cannot find symbol: method multiply(int,int)

修改:到 Calc.java 加入 multiply。一開始將測試數字寫成3*1,但容易造成return a 也通過測試，無法真正測試到 multiply(int,int)是否正確。

commit :test: multiply returns product (3,5)

commit :feat: add multiply





第 **4** 輪 **divide**:

測試是否找得到divide(7, 2)

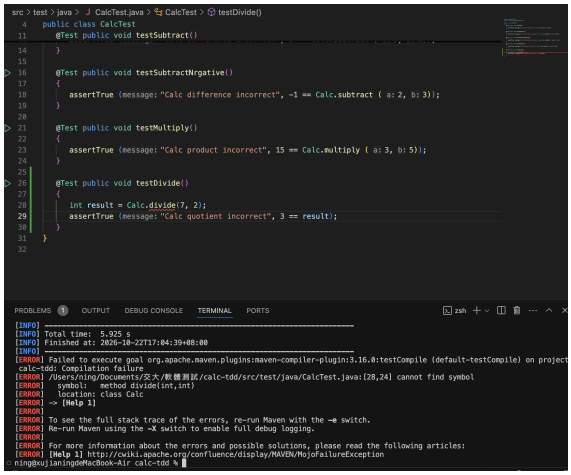
紅燈訊息：cannot find symbol: method divide(int,int)

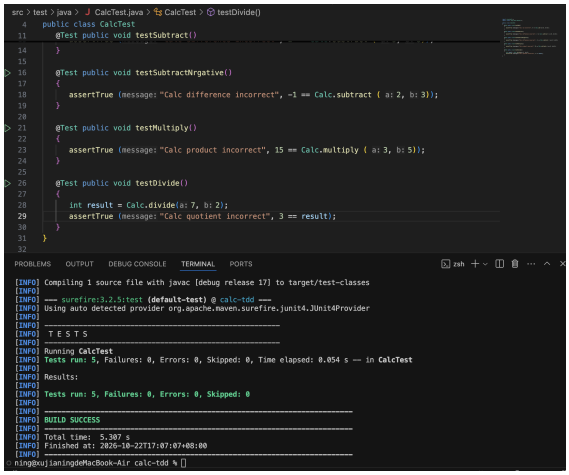
修改：一開始如同前面比較書寫test，但希望定義為回傳型別是 int，先使用

int result = Calc.divide(7, 2); 先用 int 變數接住結果再比較

commit ：test: divide returns integer quotient (7,2)

commit ：feat: add divide (integer division)





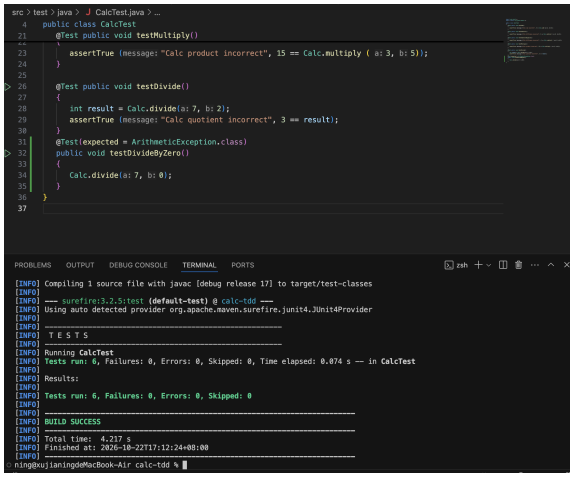
第 **5**、**6** 輪 **divide** 除以零、負數：

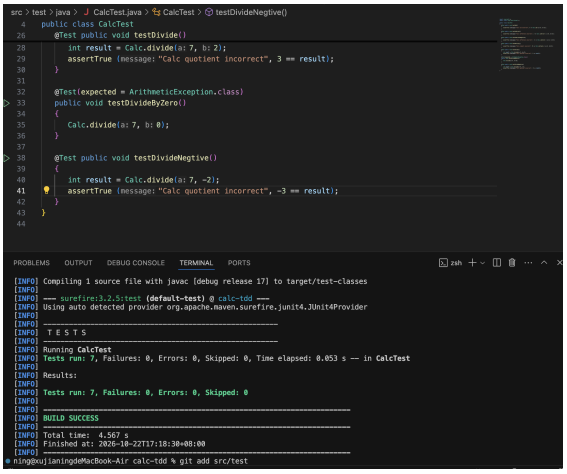
測試divide(7, 0)是否拋出ArithmeticException，divide(7, -2)的結果為-3 or -4

紅燈訊息：因java本身除以零時即會拋出ArithmeticException，不存在error，除以負數結果也正確

commit ：test: divide by zero throws ArithmeticException (passes immediately)

commit ：test: divide by negetive.





1.4 Q1 使用 AI 的範圍聲明

Q1 的測試和實作由學生撰寫，AI 沒有寫 Q1 的任何測試或 Calc 程式碼。學生在 Q1 期間向 AI 詢問過以下事項，逐字對話：

- 建置 Maven、JUnit 專案；安裝 Maven 的步驟；git 操作（`init`、`add`、`commit --amend`）。
- 用作業以外的 `max(a,b)` 範例說明紅綠重構流程，並在練習分支練習後刪除。
- 審查學生自己寫的測試並指出問題：subtract 預期值寫錯（5 應為 2）、`3×1` 分辨力不足、訊息文字誤用 sum、product、`6÷2` 分不出 int 和 double。
- 說明 JUnit 語法：`@Test(expected = ...)` 的寫法，以及用 `int result = ...` 鎖住回傳型別。

---

2. Question 6：用 AI Agent 以 TDD 加入 modulo 和 power

2.1 工作規則與證據

- Agent 只修改學生 repo 中的檔案，從未自己 **commit**。每個 test commit 都由學生審查後執行，agent 確認學生的 commit 出現在 git log 後才寫實作。
- 每輪的失敗輸出有兩份：
  - 學生在 Mac 上執行 `mvn clean test` 的輸出：`q6-evidence/cycleN-red.txt`、`cycleN-green.txt`，以及截圖 `q6-evidence/screenshots/`。
  - Agent 在自己的沙盒（OpenJDK 21 javac ＋ JUnitCore）跑的輸出：`q6-evidence/agent-run-log.txt`。
  - 第 1、2 輪學生使用截圖；第 3 輪起改為文字紀錄加截圖。

2.2 寫程式碼之前的決定

全部由學生在 agent 詢問後決定（AskUserQuestion，見對話紀錄）：

決定	學生選擇	定義它的測試	測試 **commit**
modulo 運算元為負	正負號跟被除數：`modulo(-7,3) == -1`、`modulo(7,-3) == 1`、`modulo(-7,-3) == -1`	`testModuloNegativeOperands`	`a170f27`
modulo 除以零	拋 `ArithmeticException`	`testModuloByZero`	`c6afa1a`
`power(0,0)`	`1`	`testPowerZeroToZero`	`0578a17`
負指數	整數截斷，例如 `power(2,-1) == 0`	`testPowerNegativeExponentTruncates`	`d6bf46a`

決定	學生選擇	定義它的測試	測試 commit
`power(0, 負數)`（agent 追問）	拋 `ArithmeticException`	`testPowerZeroToNegativeThrows`	`a18c04f`
溢位（agent 追問）	拋 `ArithmeticException`	`testPowerOverflowThrows`	`369b62d`

2.3 每一輪的紅綠循環

輪	Test commit	紅燈（實作前）	Feat commit	實作	綠燈
1	`c6afa1a` modulo 除以零	編譯錯誤：`cannot find symbol: method modulo(int,int)`	`340b317`	fake it：永遠 `throw new ArithmeticException`	8 tests
2	`a170f27` 負數運算元	`Errors: 1`：fake 實作拋出 `ArithmeticException: modulo by zero`	`743b568`	`return a % b;`	9 tests
3	`0578a17` `power(0,0)==1`	編譯錯誤：`cannot find symbol: method power(int,int)`	`2c47f1a`	fake it：`return 1;`	10 tests
4	`2052f7b` `power(2,3)==8`	`Failures: 1:Calc power`	`319bb4e`	迴圈連乘	11 tests

輪	Test commit	紅燈（實作前）	Feat commit	實作	綠燈
		`incorrect for (2, 3)`（回傳 1）			
5	`d6bf46a` 負指數截斷	`Failures: 1:Calc power incorrect for (2, -1)`（迴圈不執行，回傳 1）	`57c597d`	`exponent<0`：底數 ±1 特判，其餘回傳 0	12 tests
6	`a18c04f` `power(0,-1)` 拋例外	`Failures: 1:Expected exception: java.lang.ArithmeticException`	`4dc9d64`	`base==0` 時拋例外	13 tests
7	`369b62d` `power(2,31)` 拋例外	`Failures: 1:Expected exception: java.lang.ArithmeticException`（原本默默溢位成 `-2147483648`）	`cc8f1b4`	`Math.multiplyExact`	14 tests

用 git 驗證流程：每個 `test(q6)` commit 只改 `src/test/java/CalcTest.java`，每個 `feat(q6)` commit 只改 `src/main/java/Calc.java`，順序嚴格交替。Q6 期間從 `047a85a` 到 `cc8f1b4`，測試檔被刪除的行數為 0，也就是 agent 從未修改或弱化任何已 commit 的測試。驗證指令：

```
for c in $(git rev-list --reverse 047a85a..HEAD); do git show --stat --oneline $c | head -2; done

git diff 047a85a HEAD -- src/test | grep -c '^-[^-]'     # → 0
```

每輪審查意見（學生撰寫）：

- 第 1 輪：*The red bar comes from a compilation error, so the exception type has not been verified yet. I asked the agent to show, after implementing, that the test fails on an incorrect implementation that does not throw.* Agent 的回應：在沙盒中用一個 `return 0` 的錯誤版本跑測試，得到 `AssertionError: Expected exception: java.lang.ArithmeticException`，證明測試能抓到不拋例外的實作。見 `agent-run-log.txt` 的 "Cycle 1 MUTANT CHECK"。
- 第 2 到 7 輪：由ai提出程式碼及輸入prompt，我來檢察核可，輸入prompt後，給予輸出截圖讓ai確認目前進度正確

2.4 Agent 違反 TDD 規則的紀錄

#	規則	情況	嚴重度	證據
V1	(c) 決定必須在任何程式碼寫出前由測試定義	Agent 自行選擇「一個決定一輪」的順序，沒有先把四個決定測試全部寫完。因此「modulo 負數運算元」測試（`a170f27`）寫在 modulo fake 實作（`340b317`）之後；「負指數」測試（`d6bf46a`）寫在 power 實作（`2c47f1a`、`319bb4e`）之後。嚴格解讀規則 (c) 的話，這是違規。緩解：兩個測試在當時的程式上都是紅燈，表示既有程式並沒有搶先決定這些行為。	中	git 時間序：`340b317` 11:02 → `a170f27` 11:14；`319bb4e` 11:40 → `d6bf46a` 11:44
V2	(a) 先寫失敗的測試	第 1、3 輪的紅燈只是編譯錯誤，當下沒有驗證到行為規格。第 1	低	第 1 輪審查意見、

#	規則	情況	嚴重度	證據
		輪由學生指出後，agent 用錯誤實作補證明；第 3 輪沒有補。		`agent-run-log.txt`
V3	一次只寫一個失敗測試	第 2 輪一個測試方法裡有 3 個斷言，第 5 輪有 5 個。雖然仍是「一個測試方法」，但等於一次加入多個例子。前一個斷言失敗時，後面的斷言不會執行。	低（邊界）	`CalcTest.java`

沒有發生的違規（已檢查）：agent 沒有在學生 commit 測試前寫實作；沒有修改或刪除已 commit 的測試；沒有自己 commit；feat commit 都沒有碰到測試檔。

2.5 Agent 未經詢問就做的決定

#	決定	Agent 的選擇	是否事先告知	影響
D1	循環順序與策略	一個決定一輪；第 1、3 輪用 fake it（永遠拋例外、`return 1`）	實作後說明	造成 V1
D2	負指數的延伸規則	`(-2,-1) → 0`（往 0 截斷，而不是往下取整成 -1）、`(1,-3) → 1`、`(-1,-3) → -1`、`(-1,-2) → 1`	**commit** 前明確標示為 agent 推論，請學生審查；學生 commit 表示同意	規格範圍超出學生原本的選擇
D3	溢位只測一個輸入	只測 `power(2,31)`	commit 前告知這是弱點	作弊實作也能通過（Q7 證實）
D4	溢位實作	`Math.multiplyExact`	實作後說明	`power(-2,31) == Integer.MIN_`

#	決定	Agent 的選擇	是否事先告知	影響
				`VALUE` 剛好在 int 範圍內，所以不拋例外。這是實作決定的，沒有任何測試覆蓋
D5	負指數演算法	用整數特判，不用 `1.0/Math.pow`	實作後說明	沒有浮點誤差
D6	大指數效能	迴圈 O(exponent)：`power(1, Integer.MAX_VALUE)` 約需 1.4 秒	沒有考慮（Q7 才發現）	正確但很慢
D7	`modulo` 直接用 Java `%`	繼承 `Integer.MIN_VALUE % -1 == 0` 等行為	沒有	沒有測試覆蓋
D8	命名與風格	參數名 `base`、`exponent`；測試名稱；沿用 `assertTrue`；程式碼註解標記 `(agent-written)`	部分說明	無
D9	例外訊息	`"zero to a negative power"`；modulo 用 Java 內建的 `"/ by zero"`（第 1 輪自訂的 `"modulo by zero"` 在第 2 輪被取代）	說明	測試不檢查訊息

#	決定	Agent 的選擇	是否事先告知	影響
D10	證據流程	用指令把 `mvn` 輸出存成文字檔再 commit；agent 端改用 javac＋JUnitCore	有說明，學生同意	無

2.6 Q1 與 Q6 比較

速度：Q1花費時間明顯多於Q6, 且經常需要實作重複動作，效率較低，Q6思考的速度較快

測試品質: agent會主動追問`power(0,-1)` 和溢位，也同時會自行推論，思考方向較為全面，可以更快的判斷出是否執行此commit

內容一致性：不論是修改的程式或加上的commit都有一致格式，讓未來要再尋找過去的修改紀錄更為清楚

Q2、Q3：CI 與驗證工具

3.1 設定

- `.github/workflows/ci.yml`：每次 push 和 PR 時在 ubuntu-latest、Temurin JDK 17 上執行 `mvn -B verify`，並上傳 JaCoCo 報告。
- Q3 的驗證工具（寫在 `pom.xml`）：
  - **JaCoCo**：分支覆蓋率必須 100%，否則 build 失敗。原本也要求行覆蓋率 100%，但 `Calc` 的隱含建構子從未被呼叫，行覆蓋率只有 0.80，所以 `047a85a` 拿掉了行覆蓋率規則。這是覆蓋率工具抓到「從未執行的程式碼」的實例。
  - **SpotBugs**：effort=Max、threshold=Low，有發現就讓 build 失敗。學生的 Mac 是較新的 JDK，SpotBugs 4.8.6 讀不了它的 class 檔（`Unsupported class file major version`），所以本機用 `-Dspotbugs.skip=true` 跳過，改在 CI 的 JDK 17 上執行。

3.2 Q2 結果：CI 與故意弄壞 build

Repo：https://github.com/ning415415/calc-tdd Actions：https://github.com/ning415415/calc-tdd/actions

CI 時間軸（GitHub Actions，Asia/Taipei 2026-10-25）

CI	Commit	內容	結果	完成時間
#1	f5121f0	docs: Q6 evidence and Q7 property tests（第一次 push）	✅ success	09:41
#2	7c999df	chore(q2): break the build on purpose (subtract)	❌ failure	21:54
#3	015540d	fix(q2): restore subtract（學生修）	✅ success	22:08
#4	3648d1b	chore(q2): break the build again	❌ failure	22:12
#5	11831ad	fix(q2): restore multiply (agent fix)	✅ success	22:16

(0) CI 正常運作

第一次 push 後 CI #1 執行 mvn -B verify（JDK 17）：14 個測試、JaCoCo 分支覆蓋率檢查、SpotBugs 靜態分析全部通過，並上傳 jacoco-report artifact。

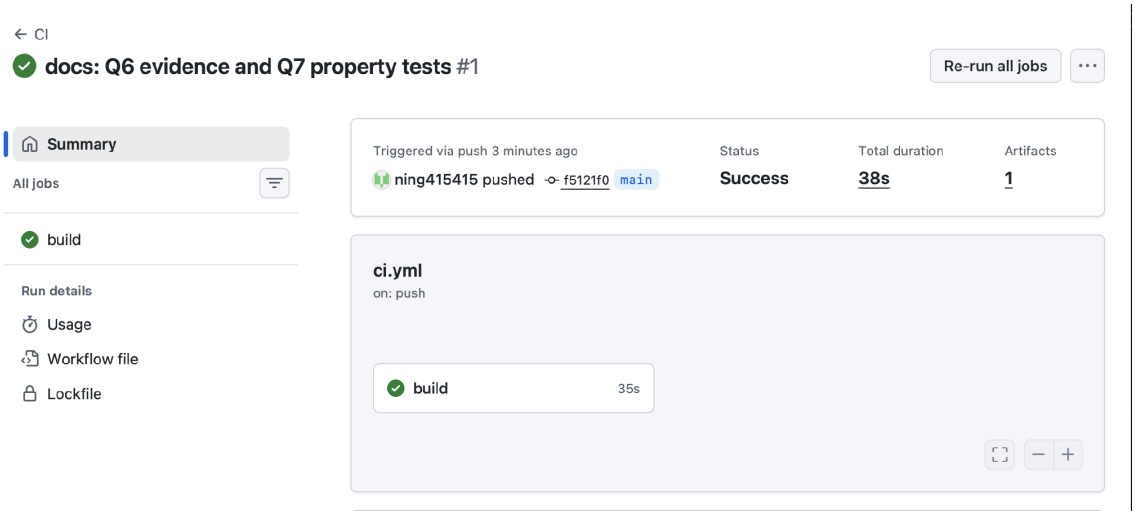


圖 *Q2-1　CI #1 綠燈（Success，38s，Artifacts 1）*

(a) 學生自己弄壞並修復

項目	內容
弄壞方式	Calc.subtract 的 return a - b; 改成 return a + b;（commit 7c999df）
CI 結果	CI #2 ❌　Tests run: 14, Failures: 2, Errors: 0 — testSubtract（CalcTest.java:13）與 testSubtractNegative（CalcTest.java:18）失敗，訊息 "Calc difference incorrect"

項目	內容
修法	改程式（Calc.java），未改測試：改回 return a - b;（commit 015540d）
驗證	CI #3 ✅
花費時間	10:00 開始 → 10:08 修好（約 8 分鐘）

我的判斷過程：
從錯誤訊息看到是testSubtract出現error ,所以先看了testSubtract所在的檔案 CalcTest.java中, testSubtract使用到了什麼函式, 看到substract函式後,再至calc.java更改 return a+b to return a-b

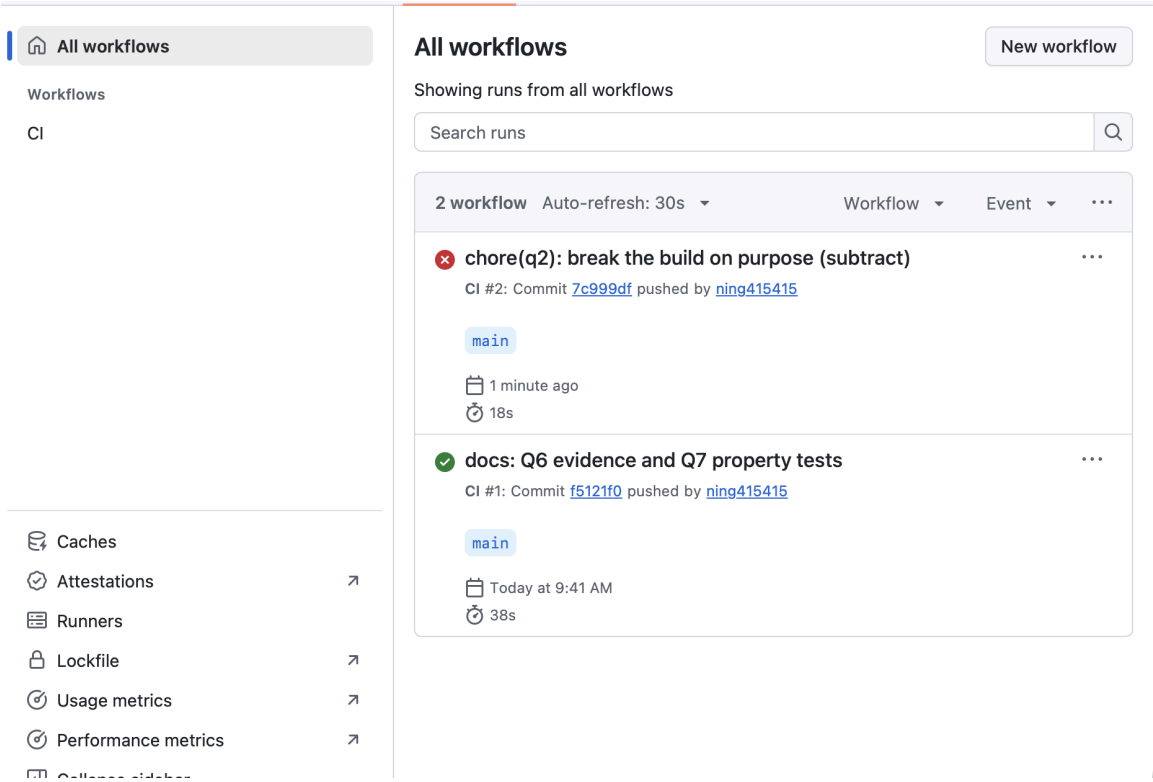


圖 *Q2-2　CI #2（7c999df）紅燈*

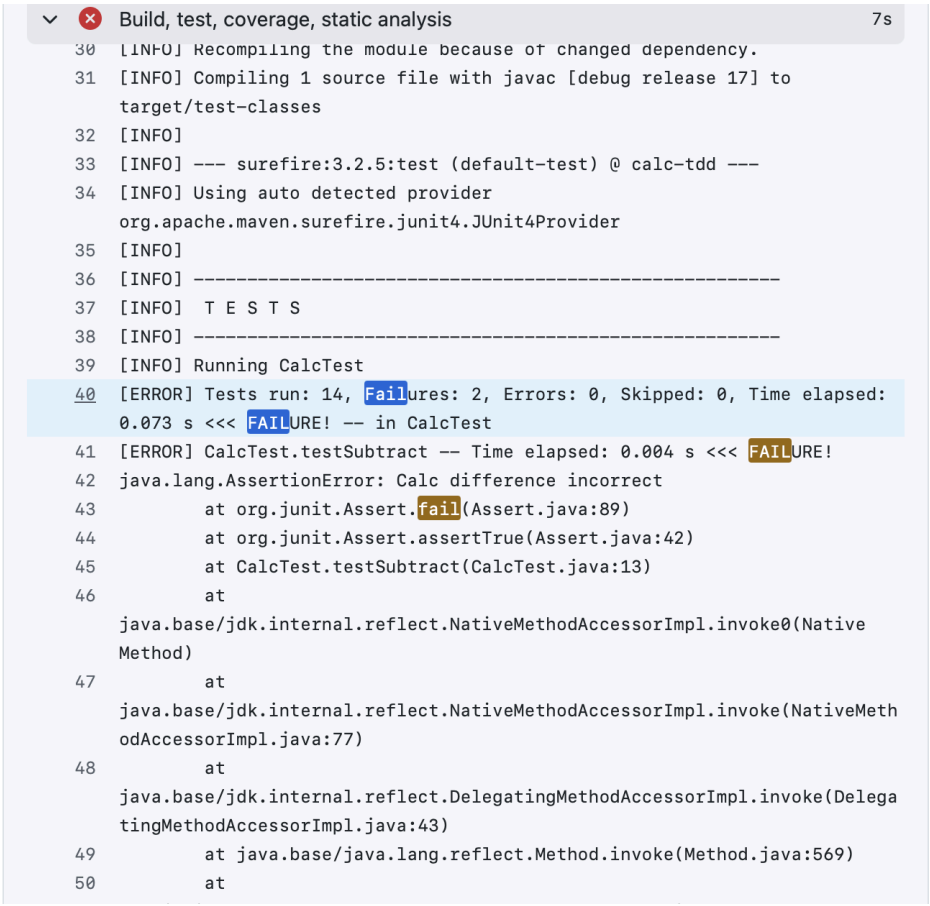


圖 *Q2-3　CI #2* 失敗細節：*testSubtract*（*CalcTest.java:13*），*Failures: 2*

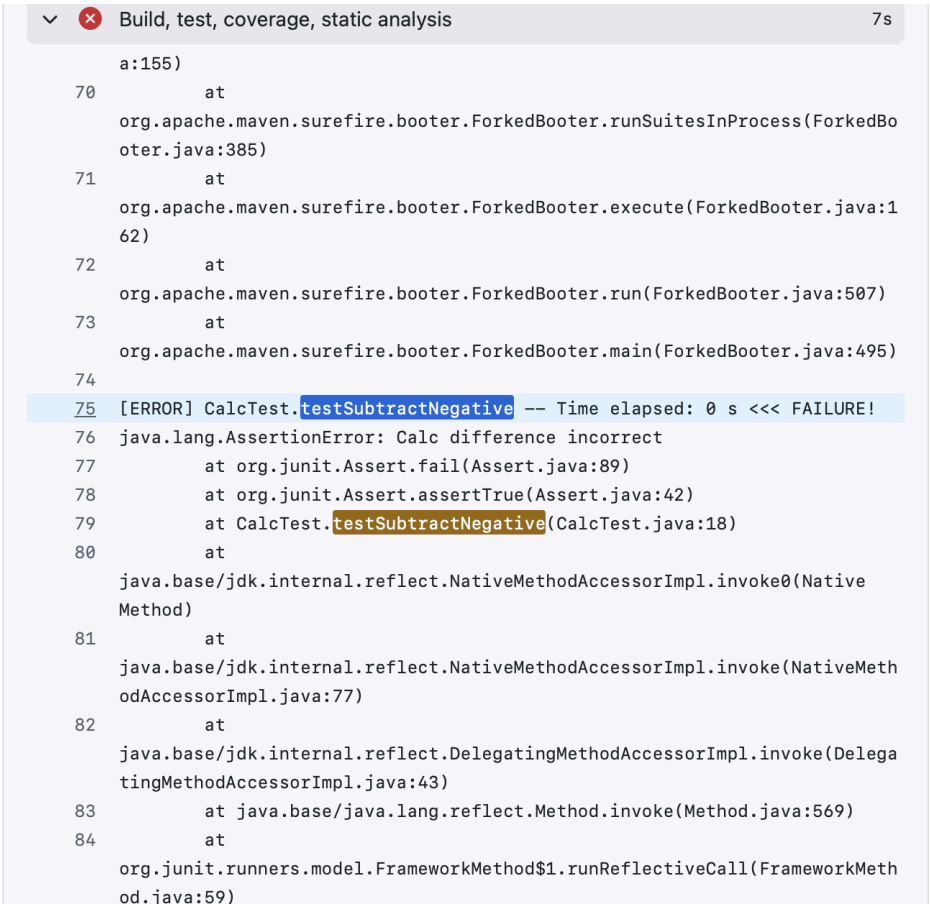


圖 *Q2-4　CI #2 失敗細節：testSubtractNegative（CalcTest.java:18）*

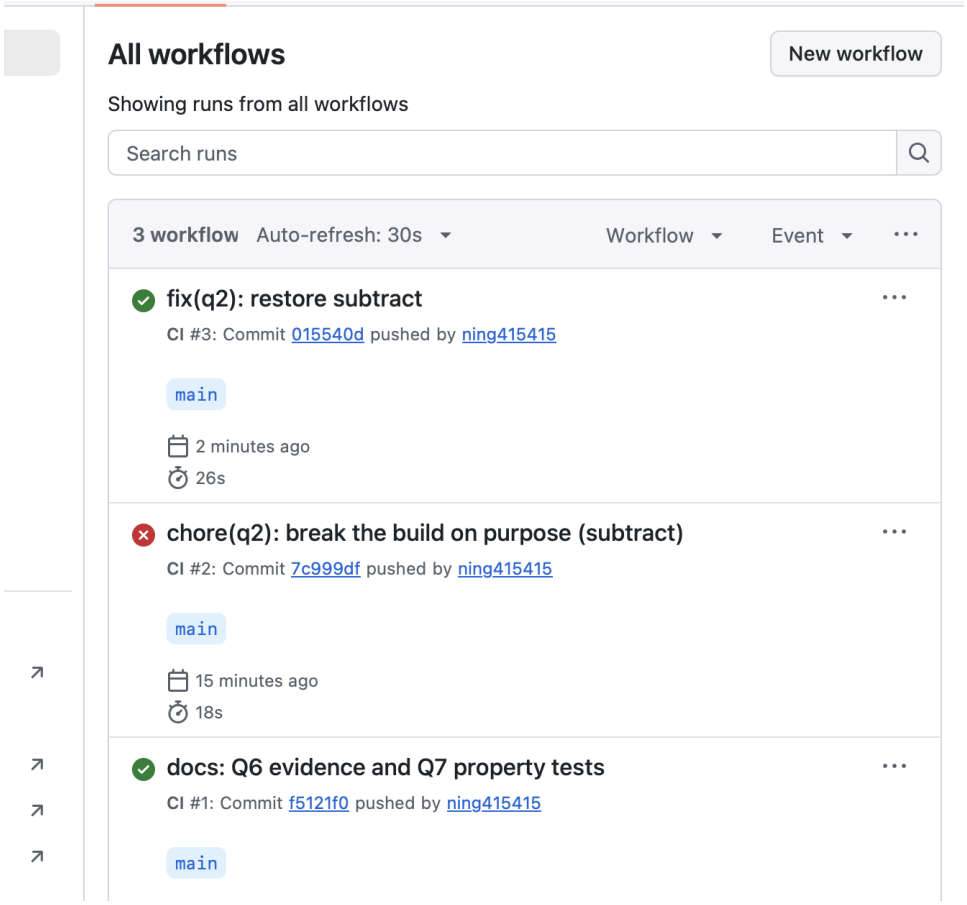


圖 *Q2-5　學生修復後 CI #3（015540d）綠燈*

(b) 再弄壞一次，交給 AI agent 修

學生另外弄壞一處、push 讓 CI #4 變紅，沒有告訴 agent 改了什麼。給 agent 的 prompt（逐字）：

```
CI 失敗了，請幫我修好讓 build 通過
```

項目	內容
弄壞方式（學生）	Calc.multiply 的 return a * b; 改成 return a / b;（commit 3648d1b）→ CI #4 ❌，testMultiply 失敗（3 / 5 = 0 ≠ 15）
Agent 的調查方式	查 CI 狀態後直接執行 git show HEAD 看最新 commit 的 diff，找到被改的那一行；再看 testMultiply 的斷言 15 == Calc.multiply(3, 5)。沒有讀 CI 的錯誤訊息。
Agent 的修法	改程式（Calc.java），未改測試：multiply 改回 return a * b;。在沙盒跑測試 OK (14 tests)。修改留在工作區，沒有自己 commit。
學生驗證與 commit	學生 git diff、mvn clean test 確認後 commit 11831ad 並 push → CI #5 ✅

項目	內容
花費時間	CI #4 紅燈 22:12 → agent 完成修改 22:13（約 1 分鐘）→ 學生 commit 22:16 → CI #5 綠燈 22:16

Agent 的修改（git diff）：

```
    static public int multiply (int a, int b)
    {
-       return a / b;
+       return a * b;
    }
```

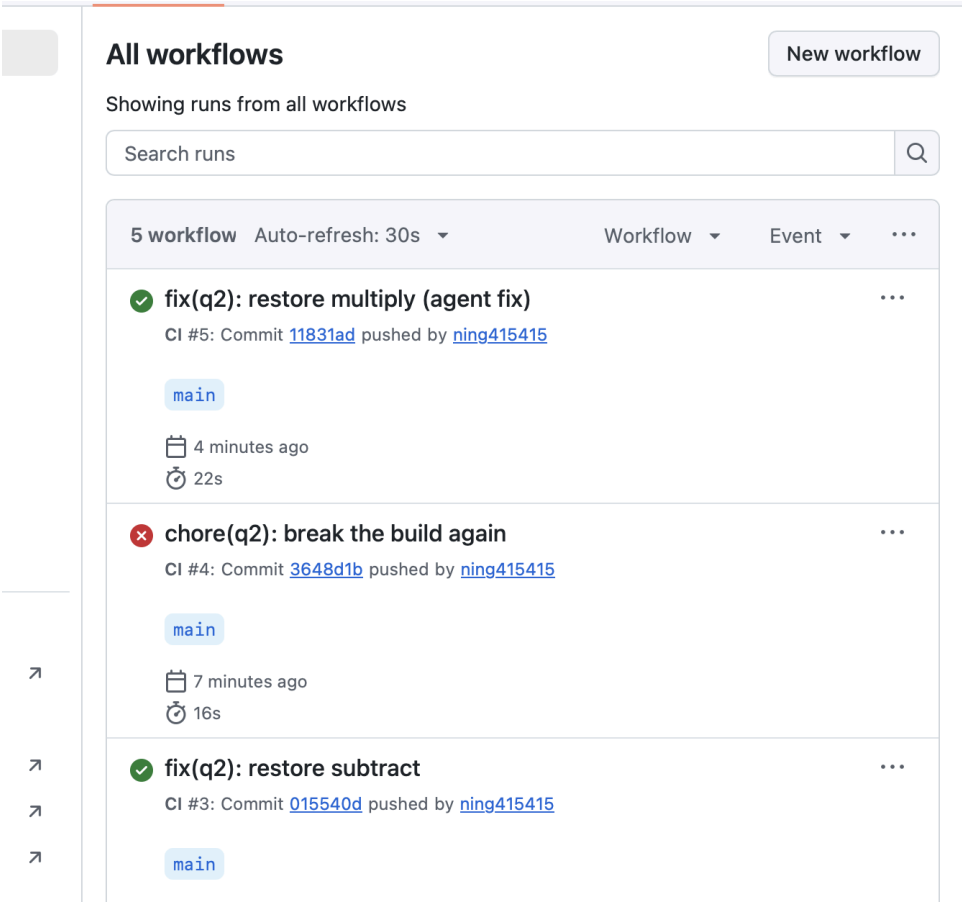


圖 *Q2-6　CI #4（3648d1b）紅燈與 agent 修復後 CI #5（11831ad）綠燈*

(c) 學生修法與 agent 修法比較

	學生（CI #2 → #3）	**Agent（CI #4 → #5）**
從哪裡開始	CI 錯誤訊息 → 測試 → 程式	最新 commit 的 diff（git show HEAD）
改了什麼	程式（subtract）	程式（multiply）

	學生（CI #2 → #3）	Agent（CI #4 → #5）
有沒有改測試	沒有	沒有
修之前有沒有問學生	—	沒有問，直接修改工作區；但沒有自己 commit
時間	約 8 分鐘	約 1 分鐘

這次弄壞的部分很顯而易見，也不需要追溯到好幾個檔案，這種情況下agent較快判斷出錯誤，但依賴diff，若是多個commit以前的，這個方式就找不到錯誤。agent 雖然有說明為什麼改程式而不是改測試，但這是它自己判斷的，沒有先詢問我測試是否才是錯的那一方。

3.3 Q3 結果：JaCoCo & SpotBugs

兩個工具都在 CI（JDK 17）的 mvn -B verify 中執行，以下證據取自 CI #1（f5121f0）。

(a) JaCoCo 覆蓋率

CI 記錄（第 578–581 行）：

```
[INFO] --- jacoco:0.8.12:check (check) @ calc-tdd ---
[INFO] Analyzed bundle 'calc-tdd' with 1 classes
[INFO] All coverage checks have been met.
```

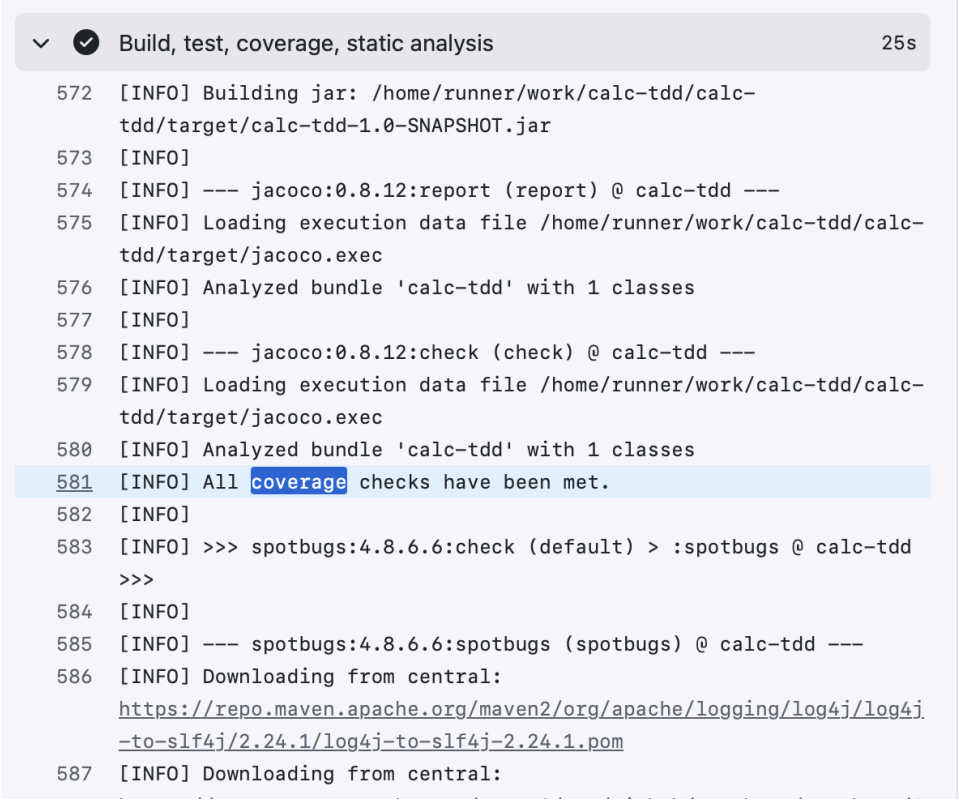


圖 *Q3-1　CI* 記錄：*JaCoCo* 檢查通過

calc-tdd

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
default		95%		100%	1	13	1	15	1	7	0	1
Total	3 of 65	95%	0 of 12	100%	1	13	1	15	1	7	0	1

Created with JaCoCo 0.8.12.202403310830

圖 *Q3-2　JaCoCo* 報告（*artifact jacoco-report*）：總覽

Calc

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
Calc()		0%		n/a	1	1	1	1	1	1
power(int, int)		100%		100%	0	7	0	9	0	1
add(int, int)		100%		n/a	0	1	0	1	0	1
subtract(int, int)		100%		n/a	0	1	0	1	0	1
multiply(int, int)		100%		n/a	0	1	0	1	0	1
divide(int, int)		100%		n/a	0	1	0	1	0	1
modulo(int, int)		100%		n/a	0	1	0	1	0	1
Total	3 of 65	95%	0 of 12	100%	1	13	1	15	1	7

Created with JaCoCo 0.8.12.202403310830

圖 *Q3-3　JaCoCo* 報告：*Calc* 各方法

指標	結果	解讀
分支覆蓋率	100%（missed 0 of 12）	power 的 12 個分支全部被測到；這是 CI 強制的門檻
指令覆蓋率	95%（missed 3 of 65）	唯一沒測到的是 Calc()，即 Java 自動產生的隱含建構子；所有方法都是 static，測試從未 new Calc()
行覆蓋率	14 / 15 行	同上，那 1 行就是建構子；這也是 commit 047a85a 拿掉行覆蓋率規則的原因
power 循環複雜度（Cxty）	7	分支最多、最需要測試的方法

```
38.   static public int power (int base, int exponent)
39.   {
40.       if (exponent < 0)
41.       {
42.           // Q6 (agent-written): 0 to a negative power is 1/0 -> student chose to throw
43.           if (base == 0)  throw new ArithmeticException("zero to a negative power");
44.           if (base == 1)  return 1;
45.           if (base == -1) return (exponent % 2 == 0) ? 1 : -1;
46.           return 0;
47.       }
48.       int result = 1;
49.       for (int i = 0; i < exponent; i++)
50.       {
51.           // Q6 cycle 7 (agent-written): multiplyExact throws ArithmeticException on any int overflow
52.           result = Math.multiplyExact(result, base);
53.       }
54.       return result;
55.   }
56. }
```

All 2 branches covered.

圖 *Q3-4　JaCoCo* 原始碼檢視：*power* 每個 *if* 都是綠色菱形（*All 2 branches covered*）

(b) SpotBugs 靜態分析

設定 effort=Max、threshold=Low（最嚴格），有任何發現就讓 build 失敗。CI 記錄（第 1263–1271 行）：

```
[INFO] Done SpotBugs Analysis....
[INFO]
[INFO] <<< spotbugs:4.8.6.6:check (default) < :spotbugs @ calc-tdd <<<
[INFO]
[INFO]
[INFO] --- spotbugs:4.8.6.6:check (default) @ calc-tdd ---
[INFO] BugInstance size is 0
[INFO] Error size is 0
[INFO] No errors/warnings found
```

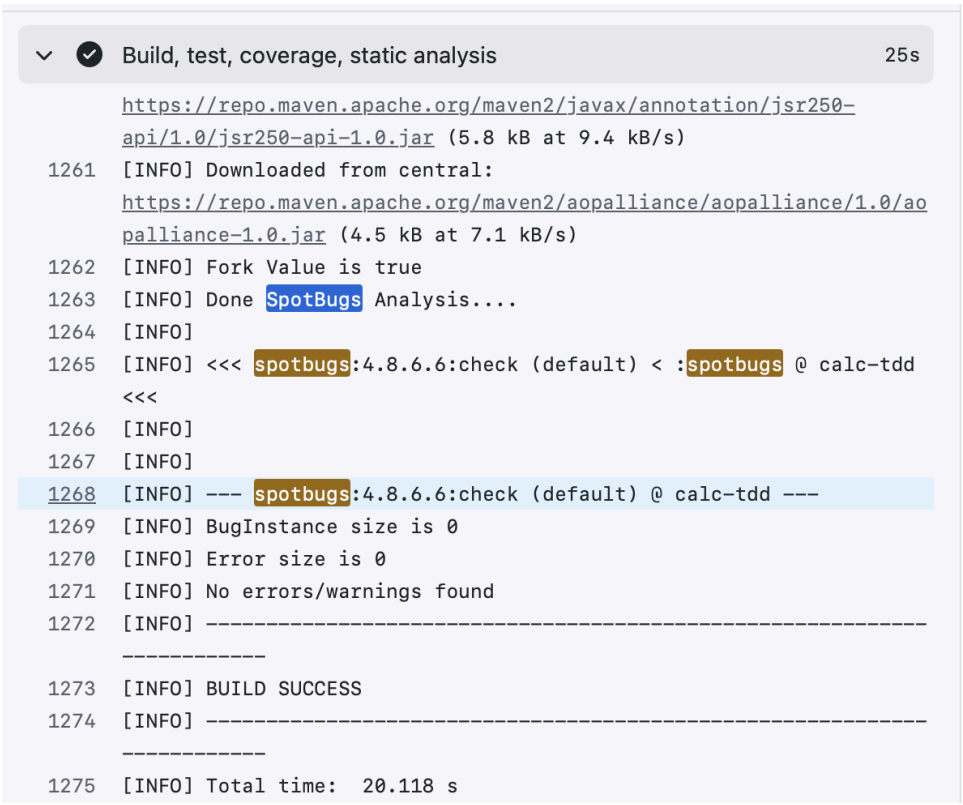


圖 *Q3-5 CI* 記錄：*SpotBugs 0* 個問題，*BUILD SUCCESS*

(c) 發現

覆蓋率工具把一行「看不見的程式碼」（隱含建構子 Calc()）標成沒測到。判斷它不影響任何行為後，選擇調整規則（只保留 100% 分支覆蓋率），而不是為了衝數字去寫 new Calc() 這種無意義的測試。SpotBugs 在最嚴格設定下沒有發現問題；學生的 Mac 因 JDK 版本較新無法執行 SpotBugs 4.8.6，所以靜態分析以 CI 的結果為準。

Q7：agent 從測試寫出的程式是否只對測試輸入有效？

4.1 方法

1. 只用測試生成 **Calc**：開一個全新的 agent，沒有任何對話脈絡，只給它 `CalcTest.java`。為了不讓它從註解讀到規則，`//` 註解全部拿掉。它要從零寫出 `Calc.java`，而且不能使用任何工具。逐字 prompt 與完整回應見 `q7/agent-transcript.md`，產出為 `q7/Calc-fresh-agent.java`。
2. 作弊對照組：手寫一個刻意「只對測試輸入有效」的 `Calc`，檔案為 `q7/Calc-overfit-strawman.java`。例如 modulo 對負數一律回傳 ±1，溢位只在 `(2,31)` 時拋例外。
3. 性質測試與蛻變測試：`q7/CalcPropertyTest.java`，共 9 個性質、上萬個隨機輸入，使用固定 seed 以確保可重現，並與 `long`、`BigInteger` 作為獨立的 oracle 比對：
   - add、subtract、multiply 與 `long` 運算一致；`subtract(add(a,b),b) == a`；交換律。
   - 除法恆等式 `divide(a,b)*b + modulo(a,b) == a`；`|modulo| < |b|`；正負號跟被除數；往 0 截斷。
   - `modulo(a,b) == modulo(a,-b)`、`modulo(-a,b) == -modulo(a,b)`。
   - 任何 a 除以零、對零取餘數都拋例外。
   - 底數 −40 到 40、指數 0 到 40 的 `power` 結果與 `BigInteger.pow` 一致；超出 int 範圍時必須拋例外，在範圍內時不能拋例外。
   - 遞迴關係 `power(b,e+1) == power(b,e)*b`。
   - 任意負指數都要截斷；`power(0, 負數)` 拋例外。
   - 極大指數，例如 `power(1, Integer.MAX_VALUE)`，要在 2 秒內完成。
4. 三個版本的 Calc 各自跑「原本 14 個範例測試」和「9 個性質測試」。

4.2 結果（`q7/q7-results.txt`）

Calc 版本	範例測試（**CalcTest**）	性質測試（**CalcPropertyTest**）
Q6 最終版（TDD＋agent）	14/14 ✅	8/9：`powerWithHugeExponentFinishesQuickly` 逾時（3 次呼叫共約 4.2 秒）
全新 agent 只看測試寫出的版本	14/14 ✅	**9/9** ✅
作弊對照組	14/14 ✅	6/9：見下方

作弊對照組被抓到的失敗：

modulo(a,b) == modulo(a,-b) for (737723,-619) expected:<1> but was:<494>

power(-40,6) = -198967296 but exact value 4096000000 does not fit in int

divide*b + modulo == a (737723,-619) q=-1191 r=1 expected:<737723> but was:<737230>

4.3 結論

1. **14** 個範例測試分辨不出「通用實作」和「只對測試輸入有效的實作」：作弊版本也能全部通過。主要原因是 modulo 只有 3 個負數例子，溢位只有 1 個例子。
2. 這次全新的 **agent** 沒有作弊：它寫出通用的 `a % b` 和 `Math.multiplyExact`，還主動列出測試沒有規範的行為，例如 add 等方法的溢位、`divide(Integer.MIN_VALUE,-1)`。它通過全部性質測試，但這只是一次觀察，不能保證每次都如此。
3. 性質測試抓到 **Q6** 版本的真實缺陷：`power` 對底數 0 和 ±1 仍然跑完整個迴圈，`power(1, Integer.MAX_VALUE)` 約需 1.4 秒。範例測試完全沒有覆蓋這種情況（見 D6）。可以再做一輪 TDD 修正：先把這個性質加入 `CalcTest` 得到紅燈，學生 commit 後，再讓 agent 實作。
4. 建議把 `CalcPropertyTest` 加入 CI，讓日後 agent 的修改不能只對範例輸入有效。

---

5. AI 使用紀錄

題目	工具	用途	逐字紀錄
Q1	Claude（Cowork）	只做說明與指導，見 §1.4；沒有寫 **Q1** 程式	AI-usage-log.pdf
Q6	Claude（Cowork），以 agent 身分直接修改 repo 檔案	寫測試與實作，學生審查並 commit	AI-usage-log.pdf；`q6-evidence/agent-run-log.txt`
Q2	Claude（Cowork）	說明 CI 設定與步驟；以 agent 身分修復學生弄壞的 build（CI #4 → #5）	AI-usage-log.pdf
Q7	Claude sub-agent（全新脈絡，未使用工具）	只用測試寫出 Calc	`q7/agent-transcript.md`（逐字 prompt ＋完整回應）

題目	工具	用途	逐字紀錄
Q7	Claude（Cowork）	撰寫性質測試、作弊對照組並執行	AI-usage-log.pdf
報告	Claude（Cowork）	整理 §0、§1.1、§1.2、§1.4、§2、§3.1、§3.2、§3.3、§4（§3.2 的判斷過程與比較文字為學生撰寫）	AI-usage-log.pdf

對話匯出：AI-usage-log.pdf。
Q6 的 prompt 大多是學生的「好了」加上測試截圖，代表「已審查並 commit」的確認。

附錄 A：全部測試

見 `src/test/java/CalcTest.java`（14 個測試：Q1 7 個、Q6 7 個）與 `q7/CalcPropertyTest.java`（9 個性質）。

附錄 B：最終版 Calc

```
public class Calc

{

  static public int add (int a, int b)      { return a + b; }

  static public int subtract (int a, int b) { return a - b; }

  static public int multiply (int a, int b) { return a * b; }

  static public int divide (int a, int b)   { return a / b; }

  // Q6 cycle 2 (agent-written)

  static public int modulo (int a, int b)

  {

    return a % b;
```

```
  }

  // Q6 cycles 3–7 (agent-written)

  static public int power (int base, int exponent)

  {

    if (exponent < 0)

    {

      if (base == 0)  throw new ArithmeticException("zero to a negative power");

      if (base == 1)  return 1;

      if (base == -1) return (exponent % 2 == 0) ? 1 : -1;

      return 0;

    }

    int result = 1;

    for (int i = 0; i < exponent; i++)

    {

      result = Math.multiplyExact(result, base);

    }

    return result;

  }

}
```

（原始檔含完整註解：`src/main/java/Calc.java`）

附錄 C：git log

11831ad fix(q2): restore multiply (agent fix)
3648d1b chore(q2): break the build again

015540d fix(q2): restore subtract
7c999df chore(q2): break the build on purpose (subtract)
f5121f0 docs: Q6 evidence and Q7 property tests
cc8f1b4 feat(q6): power throws on int overflow (Math.multiplyExact)

369b62d test(q6): power overflow throws ArithmeticException

4dc9d64 feat(q6): power(0, negative) throws ArithmeticException

a18c04f test(q6): power(0, negative) throws ArithmeticException

57c597d feat(q6): negative exponent truncates toward zero

d6bf46a test(q6): negative exponent truncates toward zero

319bb4e feat(q6): power by repeated multiplication

2052f7b test(q6): power(2,3) returns 8

2c47f1a feat(q6): add power (minimal: return 1)

0578a17 test(q6): power(0,0) returns 1

743b568 feat(q6): modulo uses Java % (sign follows dividend)

a170f27 test(q6): modulo result takes the sign of the dividend

340b317 feat(q6): add modulo (minimal: throw on any input)

c6afa1a test(q6): modulo by zero throws ArithmeticException

047a85a chore: drop line-coverage rule (implicit constructor is never called)

287265e test: divide by negetive.

846a8fa test: divide by zero throws ArithmeticException (passes immediately)

1b761b5 feat: add divide (integer division)

1b8e430 test: divide returns integer quotient (7,2)

75093c5 feat: add multiply

c72087c test: multiply returns product (3,5)

4b6acb3 refactor: split negative subtract case into its own test

f60184a test: subtract handles negative result (passes immediately)

dd5927b feat: add subtract

a5c68be test: subtract returns difference (5,3)

ae313b4 chore: starter Calc (add only) + Maven/CI setup

附錄 D：截圖索引

- Q1：截圖已內嵌於 §1.3。Q2、Q3：截圖已內嵌於 §3.2、§3.3。
- Q6：`q6-evidence/screenshots/cN-red.png`、`cN-green.png`（N = 1 到 7）、`c7-green-all-14-pass.png`（全部測試通過）、`git-log.png`